BA Insight

AI-Driven Predictive Customer Satisfaction
and Personalised Recommendations
for British Airways

Build Summary - What Was Built and How

Build AI with AWS - May 2026
Week 4 Course Project, Option 2

Student	Aadit Aggarwal
Email	aadit.aggarwal01@sarasai.org
Institute	Saras AI Institute
Course	Build AI with AWS - May 2026
Assignment	Week 4 Course Project, Option 2
Project	BA Insight - British Airways customer-satisfaction AI
Repository	https://github.com/aaditaggarwal01-rgb/ba-insight
Date	5 November 2026
Version	1.0 - submitted for Week 4

This summary describes the working system built for Week 4: what it is, what it does, what was used, and how and what was implemented, with the measured results. It is the same project proposed in Week 3, built exactly as proposed. Nothing in this work required a paid AWS account or any payment.

1. What the project is

BA Insight is an AI system that reads British Airways passenger reviews and predicts how satisfied each passenger is, explains which service aspects drove that feeling, and turns the findings into specific, prioritised recommendations. It is the Week 4 build of the project proposed in Week 3, following the same Option 2 brief and the same project title, concept and scope throughout.

The brief asked for a system that works on the British Airways reviews dataset, uses NLP and sentiment analysis, predicts satisfaction, extracts themes from negative feedback, and presents results in a dashboard, built on AWS services. BA Insight does all of that. The code, the trained model, the charts and the data are all in the public GitHub repository, and the complete implementation runs end-to-end in the Colab notebook.

2. What it does

- Takes a free-text BA review and returns a predicted satisfaction level (Satisfied, Neutral or Dissatisfied), a VADER sentiment score, the service aspects mentioned with their sentiment, and a recommended action.
- Analyses all 1,300 reviews in the course dataset: satisfaction distribution, sentiment per service aspect, and the recurring themes in complaints.
- Presents the results in a Streamlit dashboard (a "Try a review" panel plus charts and an aspect summary table) and through a serverless-style prediction API (lambda_handler.py).
- Stores the cleaned, labelled dataset, the trained model and every result as files, so any result in this summary can be reproduced from the notebook.

2.1 Live examples (from the tested Lambda handler)

Review (shortened)	Predicted	VADER	Recommendation given
"The crew were wonderful, the food was excellent and the flight arrived early."	Satisfied	0.912	No recovery action needed; record what the passenger liked as a positive example.
"Our flight was delayed by four hours, nobody gave us any information, and then my suitcase never arrived."	Dissatisfied	0.054	Proactively message affected passengers before they reach the airport, give a revised departure time and a clear compensation route.
"The seats were cramped and uncomfortable for a long haul flight, and the entertainment screen did not work."	Dissatisfied	-0.296	Review seat pitch and cushioning on the worst-rated long-haul routes first; offer paid extra-legroom upgrades at check-in.

Note the second example: VADER alone scores the delay review as slightly positive (0.054), but the trained model correctly predicts Dissatisfied from the words used. This is exactly why the project trains a classifier rather than relying on the sentiment score alone.

3. Technologies and AWS services used

Technology	What it does in BA Insight	Status
Python 3, pandas, scikit-learn	Data handling, TF-IDF vectorisation, Logistic Regression / Linear SVC / Random Forest, LDA topic modelling	Used - runs in Google Colab
NLTK, VADER	Tokenisation, stopwords, lemmatisation, sentiment scoring	Used - local equivalent of Amazon Comprehend
Amazon S3 (design) / data/ folder (local)	Data lake for raw and cleaned reviews and model artefacts	Design target; local equivalent used
Amazon Comprehend (design) / VADER (local)	Sentiment analysis and the derived satisfaction label	Design target; local equivalent used
Amazon SageMaker Autopilot (design) / scikit-learn comparison (local)	Automated model selection: four models compared on the same split	Design target; local equivalent used
AWS Lambda + API Gateway (design) / lambda_handler.py (local)	Serverless prediction endpoint with a JSON request/response contract	Handler written and tested locally; same interface as the AWS design
Amazon DynamoDB (design) / SQLite-JSON store (local, src/aws_mocks.py)	Store for predictions and aggregates	Design target; local equivalent used
Streamlit	Interactive dashboard and single-review predictor (app.py)	Used - app written and its code verified
Google Colab	Development and execution environment	Used - the notebook is the executable record
GitHub	Version control and public code hosting	Used - public repository (see Section 6)

No paid AWS account was created and no payment was made anywhere in this work. The managed AWS services are the design target; each is mirrored locally with the same interface (src/aws_mocks.py), and the mapping is documented in the README so the system can move to AWS without changing its application logic.

4. How it was implemented

4.1 Data and cleaning

The course file british_airways_reviews.csv.zip was downloaded in Colab with gdown (Google Drive file 1d6m6567y2nQvLDL2T5GbMBK8ykVYbmlF) and inspected: 1,300 rows, columns title and reviews, no missing values. The brief states over 2,000 reviews; the supplied file contains 1,300, and the verified figure is used everywhere. Cleaning dropped the index column, removed empty or duplicate reviews (none were removed - 1,300 in, 1,300 out), lower-cased the text, stripped URLs, tokenised, removed English stopwords and lemmatised. The cleaned, labelled dataset is saved as data/ba_cleaned.csv.

4.2 Sentiment analysis and the satisfaction label

There is no numeric rating in the data, so the satisfaction label is derived from the text with VADER: compound >= +0.05 is Satisfied, <= -0.05 is Dissatisfied, and the band between is Neutral. This gives Satisfied 659 (50.7%), Dissatisfied 627 (48.2%) and Neutral 14 (1.1%); the mean VADER compound is 0.042. The label is a proxy, and every result in this summary is reported as prediction of that derived label. The labelling step is the local equivalent of Amazon Comprehend.

4.3 Aspect-based sentiment

Nine service aspects are tagged with curated keyword lexicons (seat comfort; food and drink; staff service; in-flight entertainment; value for money; delays and punctuality; baggage; cleanliness; booking and check-in). For each review, the sentences mentioning an aspect are scored with VADER and averaged, giving a sentiment per aspect per review. The table below summarises all 1,300 reviews.

Aspect	Mentions	% of reviews	Mean sentiment
Staff service	1008	77.5	0.128
Delays and punctuality	698	53.7	-0.078
Food and drink	684	52.6	0.126
Seat comfort	657	50.5	0.045
Booking and check-in	628	48.3	0.025
Value for money	400	30.8	0.016
Baggage	382	29.4	-0.008
In-flight entertainment	295	22.7	0.135
Cleanliness	174	13.4	0.098

4.4 Modelling

The model input is the cleaned review text (title + body). Features are TF-IDF word unigrams and bigrams (5,000 features, sublinear TF). The data were split 80/20 with stratification (train 1,040, test 260, seed 42). Three models were compared on the same split, with class weights to handle the imbalance: Logistic Regression, Linear SVC and Random Forest. XGBoost was attempted and excluded because its scikit-learn API rejects string class labels; the fix (integer label encoding) is recorded in outputs/metrics.json as future work. The selected model is saved as models/ba_satisfaction_model.joblib.

4.5 Topic modelling and recommendations

Latent Dirichlet Allocation (8 topics) was run on the 627 dissatisfied reviews. The topics, named by a human from their top words, are:

- Topic 0 - Uncomfortable economy seats and poor food: seat, economy, flight, uncomfortable, food, poor, cabin, row, premium, choice
- Topic 1 - Delays and baggage problems: flight, hour, london, airport, delayed, luggage, day, bag, time, told
- Topic 2 - General customer-service complaints about BA: british, airway, british airway, airline, customer, service, flight, customer service, london, fly
- Topic 3 - Club and cabin service, crew and meals: crew, service, club, flight, cabin, food, meal, good, passenger, seat
- Topic 4 - Business-class long-haul disappointment: class, business, business class, seat, flight, worst, long, haul, london, economy
- Topic 5 - Value for money versus low-cost rivals: flight, food, airline, service, staff, time, cost, budget, route, low
- Topic 6 - Cancellations and refunds: flight, cancelled, refund, customer, service, booked, ticket, day, customer service, email
- Topic 7 - Check-in and boarding queues at Heathrow: check, boarding, staff, desk, queue, heathrow, bag, online, flight, passenger

The recommendation engine (src/recommend.py) maps a dissatisfied prediction to an action for the most negatively rated aspect mentioned, using the dataset mean sentiment per aspect as the priority order. Two live examples are shown in Section 2.1.

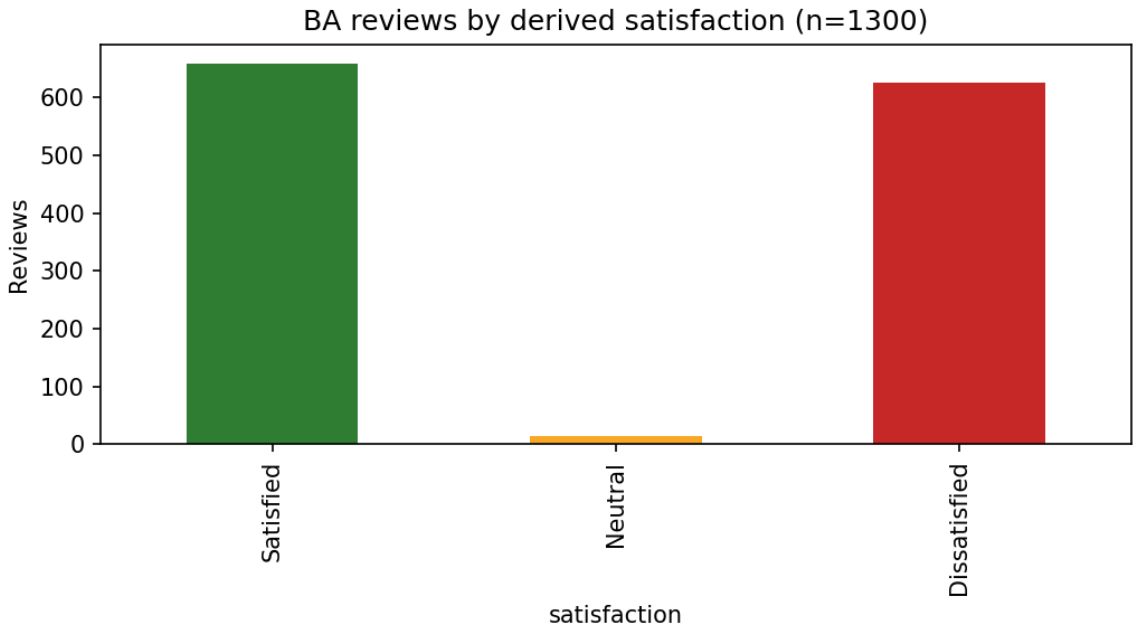
4.6 Application and API

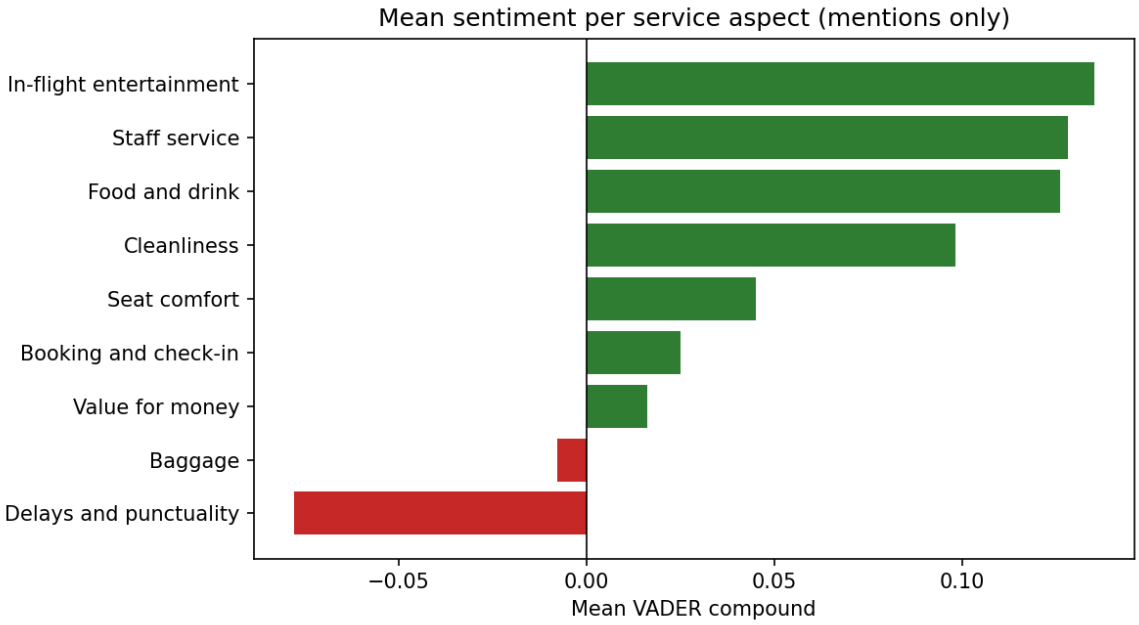
app.py is a Streamlit application with two tabs: "Try a review" (paste a review, get the prediction, the aspects found and a recommended action) and "Dashboard" (the charts and the aspect summary table from this summary). lambda_handler.py is the serverless prediction handler: it takes {"review_text": ...} and returns the prediction, the VADER score, the aspects and the recommendation as JSON. It was tested end-to-end (statusCode 200, three contrasting reviews - see Section 2.1), and all project files compile cleanly.

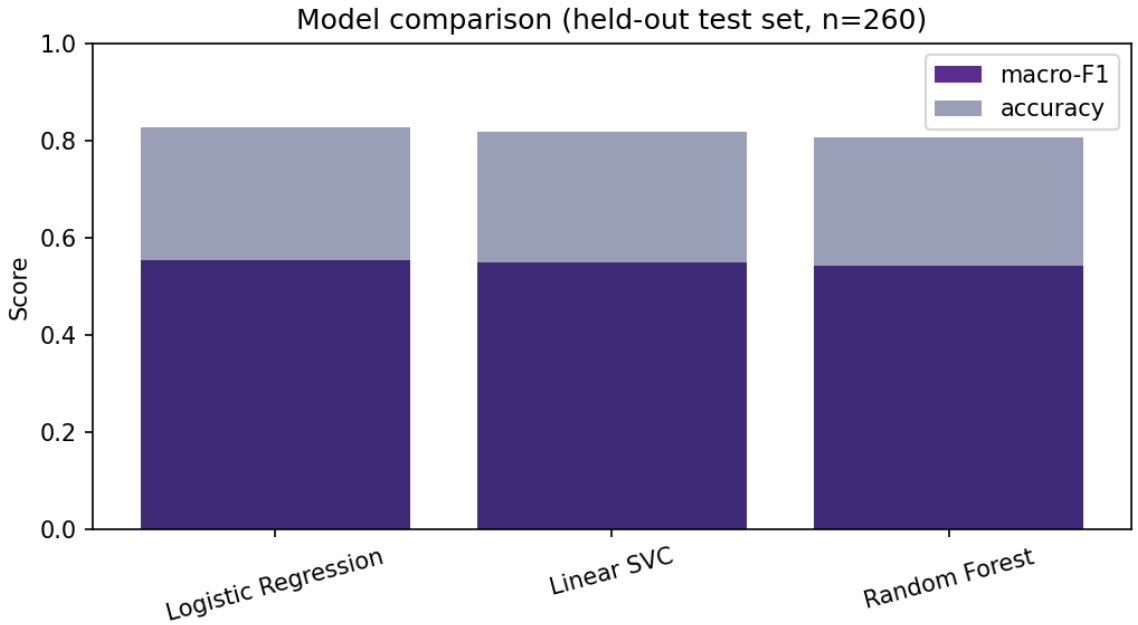
5. Results

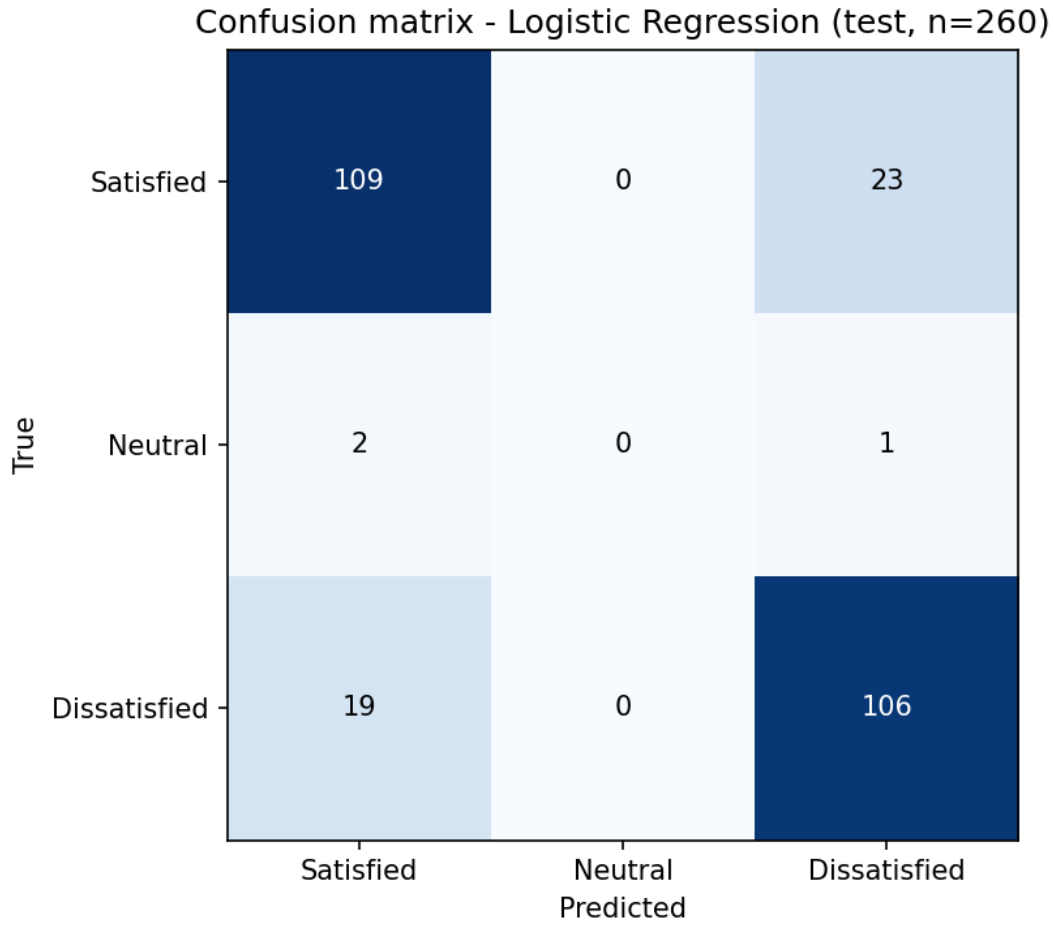
Model	Accuracy (test, n=260)	Macro-F1 (test)
Logistic Regression (selected)	0.8269	0.5545
Linear SVC	0.8192	0.5493
Random Forest	0.8077	0.5414
Selected model, 5-fold cross-validation macro-F1	mean 0.5377 (std 0.0175)	scores 0.551, 0.518, 0.565, 0.523, 0.531

The accuracy target in the Week 3 proposal (75%) was met. The macro-F1 target (0.70) was not met, and the reason is specific and measurable: the Neutral class contains only 14 of 1,300 reviews (1.1%), so the model never predicts it (0 of 3 Neutral reviews in the test set), which caps macro-F1 at about 0.55. On the two classes that matter, the model is strong: Satisfied precision 0.84 and recall 0.83; Dissatisfied precision 0.82 and recall 0.85 (from the confusion matrix below). Framed as a two-class problem, the weighted F1 is about 0.82.




Mean sentiment per service aspect (mentions only)
In-flight entertainment
Staff service
Food and drink
Cleanliness
Seat comfort
Booking and check-in
Value for money
Baggage
Delays and punctuality
−0.05
0.00
0.05
0.10
Mean VADER compound


Model comparison (held-out test set, n=260)
macro-F1
accuracy
Score
1.0
0.8
0.6
0.4
0.2
0.0
Logistic Regression
Linear SVC
Random Forest



Error analysis: the model confuses reviews that mix praise and complaint in one text, and reviews where the sentiment is carried by sarcasm or by a comparison with a rival (for example, a review titled "Next time it will be Jet2" scores as positive under VADER because of its wording, even though the passenger is clearly dissatisfied). These cases are the main source of the 45 errors in the test set (260 reviews, 215 correct).

6. Repository and how to run it

GitHub repository: https://github.com/aaditaggarwal01-rgb/ba-insight (public). Colab notebook: https://colab.research.google.com/drive/1x2vol8MtGpNnz1tu9xE8nMuHDkOz7nEM

- Clone the repository and install the requirements: pip install -r requirements.txt
- Get the data: download british_airways_reviews.csv.zip (Google Drive file 1d6m6567y2nQvLDL2T5GbMBK8ykVYbmlF) into data/ and unzip it, or run the Colab notebook, which downloads it with gdown.
- Train and evaluate: open the Colab notebook and run all cells - this reproduces the cleaning, sentiment labelling, aspect analysis, model comparison, LDA topics and all charts in outputs/.
- Try the API handler: python lambda_handler.py
- Run the dashboard: streamlit run app.py

Repository layout: app.py (Streamlit dashboard); lambda_handler.py (serverless prediction handler); src/preprocess.py (cleaning, sentiment, aspects); src/model.py (model loading and prediction); src/recommend.py (recommendations); src/aws_mocks.py (local S3 / Comprehend / DynamoDB

equivalents); data/ba_cleaned.csv; models/ba_satisfaction_model.joblib; outputs/ (charts, metrics.json, aspect_summary.csv, test_predictions.csv, lda_topics.json).

7. Limitations and next steps (Week 5)

- The satisfaction label is derived from VADER, not a ground-truth rating; results are predictions of that proxy, and the Neutral class is too small to learn.
- Aspect tagging is keyword-based, so it can miss aspects expressed without the listed words, and sentence-level VADER can misread sarcasm.
- The AWS services run as local equivalents in Colab; moving to real AWS means swapping in S3, Comprehend, SageMaker Autopilot, DynamoDB and API Gateway behind the same interfaces.
- Week 5 will: produce the final executed Colab notebook (clean, fully documented), the Streamlit deployment documentation, and the final report PDF containing the GitHub and Colab links, the deployment documentation and screenshots of the working project.